

A DSL for Optimizing Elementwise Computation in PyTorch

Nick Burgess



Motivation

- Problem: Eager Execution Overhead
 - PyTorch executes operations independently
 - Each operation launches a separate GPU kernel
 - Intermediate tensors are repeatedly written to memory
 - Overhead becomes insignificant for long elementwise chains
- Why Fusion Helps
 - Combines multiple operations into one compiled region
 - Reduces kernel launches
 - Reduces intermediate tensor allocations
 - Improves memory locality

Existing Approaches

- Existing Compiler Systems

- torch.compile
- TVM
- XLA
- Triton

- Limitations

- Fusion decisions are mostly compiler-driven
- Limited explicit user control
- Fusion behavior may be unclear in fragment or dynamic code



OpenXLA



Proposed DSL

- Lightweight Embedded DSL
 - Implemented as a Python decorator
 - Integrates directly with torch.compile
 - Preserves standard PyTorch programming model
- Key Components
 - DSL interface
 - FusionBlock wrapper
 - torch.compile backend

```
@fusion_block
def fused_chain(x):
    y = torch.sin(x)
    y = torch.cos(y)
    y = torch.exp(y)
    return torch.relu(y)
```

Experimental Setup

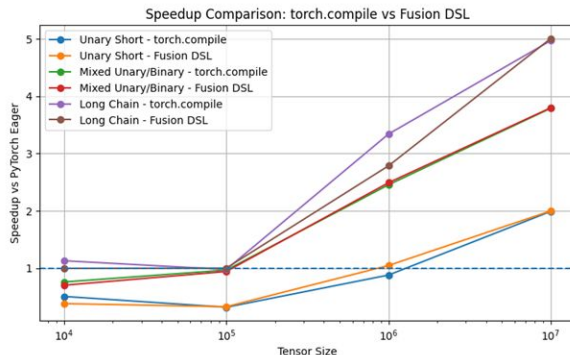
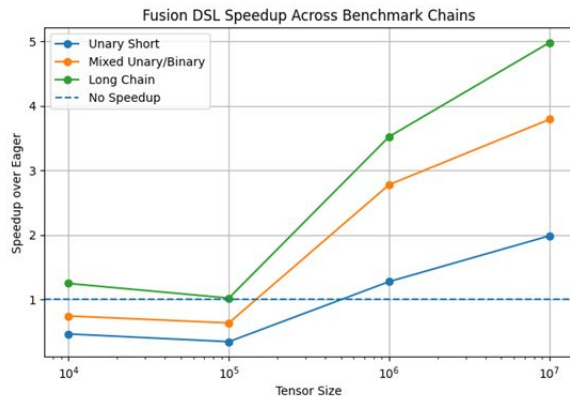
- Hardware/Software
 - Google Colab
 - NVIDIA Tesla T4 GPU
 - CUDA-enabled PyTorch
- Benchmarks
 - Unary short chain
 - Mixed unary/binary chain
 - Long elementwise chain
- Tensor Sizes
 - 10K, 100K, 1M, 10M
- Baselines
 - PyTorch eager execution
 - Direct torch.compile
 - Fusion DSL



NVIDIA
CUDA

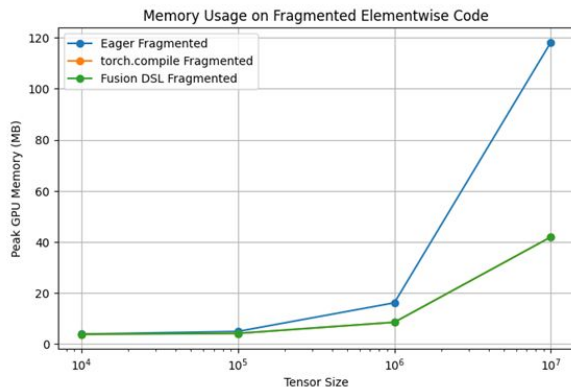
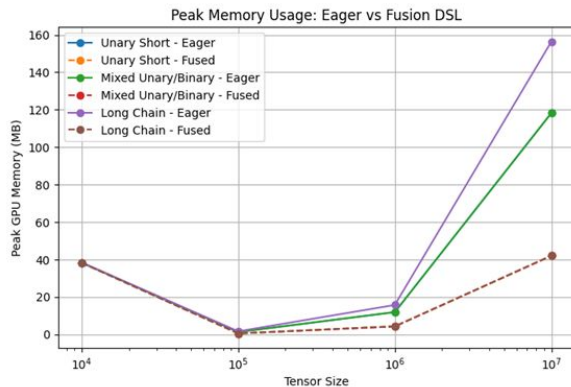
Runtime Results

- Runtime Speedup Results
 - Large tensors achieved the largest speedups
 - Long chains benefited most from fusion
 - Up to 5x speedup over eager execution



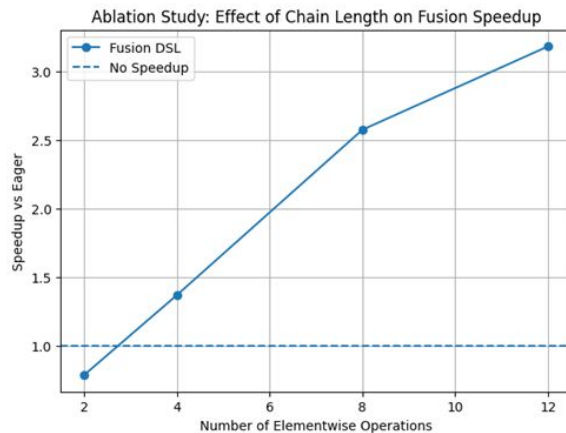
Memory Results

- Memory Usage Results
 - Fusion reduced intermediate tensor allocations
 - Significant reductions in peak GPU memory usage
 - Example: 118 MB → 42 MB



Ablation Study

- Effect of Chain Length
 - Tested chains with 2, 4, 8, and 12 operations
 - Observation: longer chains → larger speedups



	Chain Length	Eager Time (ms)	torch.compile Time (ms)	Fusion DSL Time (ms)	DSL Speedup vs Eager	Correct
0	2	0.069708	0.098549	0.088483	0.787806	True
1	4	0.136149	0.092599	0.099388	1.369873	True
2	8	0.276706	0.105468	0.107529	2.573308	True
3	12	0.417173	0.131099	0.131193	3.179829	True

Discussion

- Key Takeaways

- Fusion significantly improves runtime and memory efficiency
- Longer chains benefit most
- DSL and torch.compile produced similar performance

- Limitations

- Only supports elementwise operations
- No custom kernel generation
- Evaluated only on Tesla T4 GPU

Conclusion

- Conclusion
 - Lightweight DSL for explicit fusion regions
 - Preserves PyTorch programming model
 - Up to 5x runtime speedup
 - Significant GPU memory reductions
- Future Work
 - Reductions and matrix operations
 - Larger neural network workloads
 - Custom kernel generation
 - Triton integration